

WEEK - 1

PLSQL_Exercises

Exercise 1: Control Structures

```
[ SQL Worksheet ]*  ▶  ⌵  🔗  📄  ⌵  Aa  🗑️  📤  📄

1  CREATE TABLE Customers (
2      CustomerID NUMBER PRIMARY KEY,
3      CustomerName VARCHAR2(50),
4      Age NUMBER,
5      Balance NUMBER,
6      IsVIP VARCHAR2(5)
7  );
8
9  CREATE TABLE Loans (
10     LoanID NUMBER PRIMARY KEY,
11     CustomerID NUMBER,
12     InterestRate NUMBER,
13     DueDate DATE,
14     FOREIGN KEY (CustomerID)
15     REFERENCES Customers(CustomerID)
16 );
```

```
[ SQL Worksheet ]*  ▶  ⌵  🔗  📄  ⌵  Aa  🗑️  📤  📄

1  INSERT INTO Customers VALUES (1, 'Rahul', 65, 15000, 'FALSE');
2  INSERT INTO Customers VALUES (2, 'Priya', 45, 8000, 'FALSE');
3  INSERT INTO Customers VALUES (3, 'Amit', 70, 20000, 'FALSE');
4  INSERT INTO Customers VALUES (4, 'Sneha', 30, 12000, 'FALSE');
5
6  INSERT INTO Loans VALUES (101, 1, 10, SYSDATE + 15);
7  INSERT INTO Loans VALUES (102, 2, 12, SYSDATE + 45);
8  INSERT INTO Loans VALUES (103, 3, 11, SYSDATE + 20);
9  INSERT INTO Loans VALUES (104, 4, 9, SYSDATE + 10);
10
11 COMMIT;
```

WEEK - 1

Query result						Script output	DBMS output	Explain Plan	SQL history
Download						Execution time: 0.006 seconds			
	LOANID	CUSTOMERID	INTERESTRATE	DUEDATE					
1	101	1	9	7/7/2026, 10:20:55					
2	102	2	12	8/6/2026, 10:20:55					
3	103	3	10	7/12/2026, 10:20:55					
4	104	4	9	7/2/2026, 10:20:55					

Query result							Script output	DBMS output	Explain Plan	SQL history
Download							Execution time: 0.011 seconds			
	CUSTOMERID	CUSTOMERNAME	AGE	BALANCE	ISVIP					
1	1	Rahul	65	15000	TRUE					
2	2	Priya	45	8000	FALSE					
3	3	Amit	70	20000	TRUE					
4	4	Sneha	30	12000	TRUE					

WEEK - 1

Scenario 1: Apply 1% Discount to Senior Citizens

```
BEGIN
  FOR rec IN (
    SELECT c.CustomerID, l.LoanID, l.InterestRate
    FROM Customers c
    JOIN Loans l
    ON c.CustomerID = l.CustomerID
    WHERE c.Age > 60
  )
  LOOP
    UPDATE Loans
    SET InterestRate = InterestRate - 1
    WHERE LoanID = rec.LoanID;

    DBMS_OUTPUT.PUT_LINE(
      'Discount applied to Customer ID '
      || rec.CustomerID
    );
  END LOOP;

  COMMIT;
END;
```

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> BEGIN
      FOR rec IN (
        SELECT c.CustomerID, l.LoanID, l.InterestRate
        FROM Customers c...
```

Show more...

Discount applied to Customer ID 1
Discount applied to Customer ID 3

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.169

WEEK - 1

Scenario 2: Promote Customers to VIP

```
[ SQL Worksheet ]*  ▶  ≡  🔗  📄  ≡  Aa  🗑️  📤  📖

1  BEGIN
2      FOR rec IN (
3          SELECT CustomerID
4          FROM Customers
5          WHERE Balance > 10000
6      )
7      LOOP
8          UPDATE Customers
9          SET IsVIP = 'TRUE'
10         WHERE CustomerID = rec.CustomerID;
11
12         DBMS_OUTPUT.PUT_LINE(
13             'Customer '
14             || rec.CustomerID
15             || ' promoted to VIP.'
16         );
17     END LOOP;
18
19     COMMIT;
20 END;
21 /
```

Query result Script output DBMS output Explain Plan SQL history

🗑️ 📄

FOR rec IN (
SELECT CustomerID
FROM Customers...

Show more...

Customer 1 promoted to VIP.
Customer 3 promoted to VIP.
Customer 4 promoted to VIP.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.164

WEEK - 1

Scenario 3: Send Loan Due Reminders

```
[ SQL Worksheet ]*  [Run] [Format] [Find] [Save] [Undo] [Redo] Aa [Clear] [Help]

1  SET SERVEROUTPUT ON;
2
3  BEGIN
4      FOR rec IN (
5          SELECT c.CustomerName,
6                 l.LoanID,
7                 l.DueDate
8          FROM Customers c
9          JOIN Loans l
10         ON c.CustomerID = l.CustomerID
11         WHERE l.DueDate <= SYSDATE + 30
12     )
13     LOOP
14         DBMS_OUTPUT.PUT_LINE(
15             'Reminder: Dear '
16             || rec.CustomerName
17             || ', Loan ID '
18             || rec.LoanID
19             || ' is due on '
20             || TO_CHAR(rec.DueDate, 'DD-MON-YYYY')
21         );
22     END LOOP;
23 END;
```

Query result **Script output** DBMS output Explain Plan SQL history

 

```
FOR rec IN (
  SELECT c.CustomerName,
         l.LoanID,...
```

Show more...

Reminder: Dear Rahul, Loan ID 101 is due on 07-JUL-2026
Reminder: Dear Amit, Loan ID 103 is due on 12-JUL-2026
Reminder: Dear Sneha, Loan ID 104 is due on 02-JUL-2026

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.041

WEEK - 1

Exercise 2: Stored Procedures

```
[ SQL Worksheet ]*  ▶  ≡  🔗  📄  ≡  Aa  ▼  🗑️  📤  🔗

1  CREATE TABLE Accounts (
2      AccountID NUMBER PRIMARY KEY,
3      CustomerName VARCHAR2(50),
4      AccountType VARCHAR2(20),
5      Balance NUMBER(10,2)
6  );
7
8  CREATE TABLE Employees (
9      EmployeeID NUMBER PRIMARY KEY,
10     EmployeeName VARCHAR2(50),
11     Department VARCHAR2(30),
12     Salary NUMBER(10,2)
13 );
```

```
Query result  Script output  DBMS output  Explain Plan  SQL history

🗑️  📄

SQL> CREATE TABLE Accounts (
      AccountID NUMBER PRIMARY KEY,
      CustomerName VARCHAR2(50),
      AccountType VARCHAR2(20),...
Show more...

Table ACCOUNTS created.
Elapsed: 00:00:00.033

SQL> CREATE TABLE Employees (
      EmployeeID NUMBER PRIMARY KEY,
      EmployeeName VARCHAR2(50),
      Department VARCHAR2(30),...
Show more...

Table EMPLOYEES created.
Elapsed: 00:00:00.021
```

WEEK - 1

[SQL Worksheet]*

```
1  INSERT INTO Accounts VALUES (101, 'Rahul', 'Savings', 50000);
2  INSERT INTO Accounts VALUES (102, 'Priya', 'Savings', 30000);
3  INSERT INTO Accounts VALUES (103, 'Amit', 'Current', 40000);
4  INSERT INTO Accounts VALUES (104, 'Sneha', 'Savings', 60000);
5
6  INSERT INTO Employees VALUES (1, 'Raj', 'IT', 50000);
7  INSERT INTO Employees VALUES (2, 'Anita', 'HR', 45000);
8  INSERT INTO Employees VALUES (3, 'Karan', 'IT', 60000);
9  INSERT INTO Employees VALUES (4, 'Meera', 'Finance', 55000);
10
11 COMMIT;
```

Table EMPLOYEES created.

Elapsed: 00:00:00.021

```
SQL> INSERT INTO Accounts VALUES (101, 'Rahul', 'Savings', 50000)
```

1 row inserted.

Elapsed: 00:00:00.023

```
SQL> INSERT INTO Employees VALUES (3, 'Karan', 'IT', 60000)
```

1 row inserted.

Elapsed: 00:00:00.003

```
SQL> INSERT INTO Employees VALUES (4, 'Meera', 'Finance', 55000)
```

1 row inserted.

Elapsed: 00:00:00.003

```
SQL> COMMIT
```

WEEK - 1

Scenario 1: Process Monthly Interest

[SQL Worksheet]* ▼



Aa ▼



```
1 CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
2 AS
3 BEGIN
4     UPDATE Accounts
5     SET Balance = Balance + (Balance * 0.01)
6     WHERE AccountType = 'Savings';
7
8     COMMIT;
9
10    DBMS_OUTPUT.PUT_LINE(
11        'Monthly interest processed successfully.'
12    );
13 END;
```

Commit complete.

Elapsed: 00:00:00.002

```
SQL> CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
    UPDATE Accounts...
```

Show more...

Procedure PROCESSMONTHLYINTEREST compiled

Elapsed: 00:00:00.028

Query result

Script output

DBMS output

Explain Plan

SQL history



Download ▼

Execution time: 0.01 seconds

	ACCOUNTID	CUSTOMERNAME	ACCOUNTTYPE	BALANCE	
1	101	Rahul	Savings	51005	
2	102	Priya	Savings	30603	
3	103	Amit	Current	40000	
4	104	Sneha	Savings	61206	

WEEK - 1

Scenario 2: Update Employee Bonus

```
[ SQL Worksheet ]*  ▶  ≡  🔑  ↻  ≡  Aa  🗑️  ⬆️  🔗

1  CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
2      p_department IN VARCHAR2,
3      p_bonus IN NUMBER
4  )
5  AS
6  BEGIN
7      UPDATE Employees
8      SET Salary = Salary + (Salary * p_bonus / 100)
9      WHERE Department = p_department;
10
11     COMMIT;
12
13     DBMS_OUTPUT.PUT_LINE(
14         'Bonus updated successfully.'
15     );
16 END;
17 /
```

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.009 seconds

	EMPLOYEEID	EMPLOYEENAME	DEPARTMENT	SALARY
1	1	Raj	IT	55000
2	2	Anita	HR	45000
3	3	Karan	IT	66000
4	4	Meera	Finance	55000

WEEK - 1

Scenario 3: Transfer Funds

```
[ SQL Worksheet ]*      Aa    
```

```
1 CREATE OR REPLACE PROCEDURE TransferFunds(  
2     p_fromAccount IN NUMBER,  
3     p_toAccount IN NUMBER,  
4     p_amount IN NUMBER  
5 )  
6 AS  
7     v_balance NUMBER;  
8 BEGIN  
9  
10    SELECT Balance  
11    INTO v_balance  
12    FROM Accounts  
13    WHERE AccountID = p_fromAccount;  
14  
15    IF v_balance >= p_amount THEN  
16  
17        UPDATE Accounts  
18        SET Balance = Balance - p_amount  
19        WHERE AccountID = p_fromAccount;  
20  
21        UPDATE Accounts  
22        SET Balance = Balance + p_amount  
23        WHERE AccountID = p_toAccount;  
24  
25        COMMIT;  
26  
27        DBMS_OUTPUT.PUT_LINE(  
28            'Transfer completed successfully.'  
29        );  
30  
31    ELSE  
32  
33        DBMS_OUTPUT.PUT_LINE(  
34            'Insufficient balance.'
```

Query result					
Script output					
DBMS output					
Explain Plan					
SQL history					
  Download Execution time: 0.001 seconds					
	ACCOUNTID	CUSTOMERNAME	ACCOUNTTYPE	BALANCE	
1	101	Rahul	Savings	41005	
2	102	Priya	Savings	40603	
3	103	Amit	Current	40000	
4	104	Sneha	Savings	61206	

WEEK - 1